Fine-tuning MACE with the mdstats campaign CLI

The campaign CLI turns source-certified VASP data into a current selected training set and a fresh production model. It keeps scientific authorities, restart state, replay identity, checkpoint evidence, and currentness checks in one disk-backed campaign.

Run commands from the repository root:

```
python tools/mdstats-mlff-campaign.py --config campaign.toml <command>
```

Current workflow

The public scientific lifecycle is exactly:

```
init -> doctor -> prepare -> select-target-size -> cross-validate -> train-production
```

Post-production qualification of the finished product is a separate downstream family:

```
qualification status | qualification run | qualification activate-locked
```

`storage` is an orthogonal artifact-management command whose eligibility comes from the real owners, not from pathnames. `status` reports the whole lifecycle through qualification, so a frozen but unqualified product is never described as a finished campaign, and `advance` chooses the next safe owner from that lifecycle. Neither introduces another scientific state machine. `advance` may run ordinary `qualification run`; it can never open locked evidence, because that disclosure is irreversible and stays an explicit operator command.

`status` and `qualification status` are observational in the strict sense: they resolve paths without creating them, open the campaign database read-only, and report what durable evidence says. They construct no trainer, provider, session, or evidence directory, so running them repeatedly leaves the workspace byte-identical. What they cannot know cheaply they say plainly rather than computing it - a full re-authentication is the consequential command's job.

1. Create a configuration

```
python tools/mdstats-mlff-campaign.py --config campaign.toml init
```

Set the paths and foundation identity in the generated file. A minimal campaign provides a training root, an inspected foundation checkpoint, and one selected replay corpus:

```
[paths]
training_root = "/path/to/LTA_training"
foundation_model = "/path/to/mace-foundation.model"
replay_set = "/path/to/replay.extxyz"
```

The replay label policy is explicit in the generated file and defaults to the source DFT labels:

```
[replay]
label_mode = "true_dft"
```

`true_dft` trains the replay head on the source labels and runs no foundation inference during `prepare`. `foundation_pseudolabel` is an explicit opt-in: `prepare` then owns one replay-wide foundation inference pass - potentially long and VRAM-heavy - and an independent TRUE_DFT monitor is still mandatory. Replay pseudo labels are never a silent fallback for missing source labels, and `prediction_batch_size`/`prediction_shard_size` are execution realization only: editing them never causes reinference.

The generator exposes the current target-size policy explicitly:

```
[target_data.size_convergence]
target_size_power_min = 7
target_size_power_max = 14
```

```
evaluation_size_powers = [8, 9, 10]
fidelity_epochs = [1, 3, 10]
```

The power ceiling is configuration, not a hidden fixed-size scientific limit. The available population still bounds materializable candidates. Optimizer seeds are authored only by the sole enabled training method.

Current post-selection CV settings are authored under one canonical section:

```
[post_selection.cv]
fold_count = 2
partition_seed = 7
seeds = [11]
max_num_epochs = 2
acceptance_maximum = 0.5

[post_selection.production]
seeds = [5]
```

Pre-target fold controls are not generated as target-size authority. Historical read-only fields may be accepted for a compatible existing campaign, but they cannot silently become current scientific settings.

The learned model supports `single` (FP32) or `double` (FP64) precision. mdstats scientific reductions, reference fitting, geometry, and persistent bookkeeping remain FP64 in either mode. The selected acceleration backend and MACE runtime identity are bound by `doctor` and the campaign protocol.

Shared optimizer settings, and what they do and do not control

The shared scientific optimizer settings are authored once under `[training]`:

```
[training]
learning_rate = 1.0e-4
batch_size = 2
valid_batch_size = 2
eval_interval = 1
ema = true
ema_decay = 0.99999
amsgrad = true
weight_decay = 1.0e-6
clip_grad = 10.0
```

They are resolved once and feed both the recorded post-selection method identity and the optimizer that actually trains, so an explicit value here always changes both, and an omitted key resolves to the same default on both sides. The recorded method is therefore the method that ran. Evidence produced under the older resolution - where identity and execution could default differently - is kept as history and cannot authorize corrected cross-validation or final production.

Three things under `[training]` are deliberately *not* shared method settings:

- `num_workers` is pure resource scheduling. It is not method identity and not target-size scientific currentness, so it can be retuned between runs, even mid-screen, without invalidating anything.
- `max_num_epochs` is the final-production horizon. The target-size screen uses its own `fidelity_epochs` schedule, and cross-validation uses `[post_selection.cv].max_num_epochs`.
- `learning_rate` and `ema_decay` are the post-selection/general training authority. The target-size screen derives its own effective learning rate and EMA decay from `[target_data.size_convergence.optimizer_normalization]` and the candidate's `ceil(N/batch_size)` update geometry; editing the general values cannot change a screen result.

`batch_size` and the learned-model precision *do* change target-size training execution, so editing them retires P3 screen evidence - but not the prepared P1/P2 statistical substrate or the common fit, which consume neither. The harness-validation `valid_batch_size` is fixed non-controlling validation geometry and, like `num_workers`, may drift mid-screen.

2. Check inputs and runtime

```
python tools/mdstats-mlff-campaign.py --config campaign.toml doctor
```

Do not continue until blocking source, manifest, foundation, replay, backend, and runtime checks pass. `doctor` records the manifest approval and the exact runtime realization used by later owners. A missing accelerator or unavailable long-production environment is reported as unavailable; it is not converted into a qualification pass.

For a single-source replay campaign `doctor` validates replay prerequisites only. It does not build the prediction cache, run replay inference, or publish replay records, and it says so: prediction-dependent replay eligibility and qualification are deferred to `prepare`. If `doctor` is fast on a `foundation_pseudolabel` campaign, that is correct - the replay-wide inference pass belongs to `prepare`, which reports whether it reused an authenticated prediction cache, built a new one, or rebuilt an invalid one.

3. Prepare the current substrate

Review the source manifest first:

```
python tools/mdstats-mlff-campaign.py --config campaign.toml prepare
```

When the manifest needs operator approval, approve that exact digest:

```
python tools/mdstats-mlff-campaign.py --config campaign.toml prepare --approve-manifest
python tools/mdstats-mlff-campaign.py --config campaign.toml prepare
```

Use `--refresh-inferences` before approval when source metadata or inference inputs changed. Preparation is restartable and owns only the current neutral substrate:

```
source/frame/label authority
  -> protected statistical relations
  -> one P_train/M3 split and pi_train/pi_eval
  -> one common target-size preparation
```

`prepare` does not select a target size, train a candidate, rank a checkpoint, or publish production. The configured ladder is an experiment definition, not a decision. The one-time destructive cutover rejects obsolete derived target-size records before reuse and quarantines them rather than migrating them. Those records are never translated into current authority.

Preparation reuses only lower-level inputs that current owners revalidate. A source or scientific-identity change invalidates the affected generation; provenance-only presentation changes do not change arithmetic. Interrupted work is resumed by rerunning the same command after inspecting `status`.

4. Choose the target size and the training horizons

This step is a decision *you* make. The command records it; nothing here freezes it.

```
# choose a size explicitly - runs no training at all
python tools/mdstats-mlff-campaign.py --config campaign.toml select-target-size 512

# or run (or reuse) the optional automatic diagnostic and take its recommendation
python tools/mdstats-mlff-campaign.py --config campaign.toml select-target-size --auto
```

A bare `select-target-size` is invalid, and `<N>`, `--auto` and `--reset` cannot be combined. `<N>` must be one of the configured qualified candidate sizes, which `status` lists. Every candidate is an exact prefix of one deterministic `pi_train` order, so the chosen set is always

```
T_N = pi_train[:N]
```

There is no other membership constructor: no resampling, no arbitrary list.

Selecting several sizes

`prepare` is expensive and its result is deliberately reusable, so you can ask for **more than one** size from the same prepared generation and compare how the downstream experiment behaves. The design is an ordered list of distinct sizes:

```
select-target-size 512          # design is [512]
select-target-size 1024         # design is [512, 1024]   <- appended, not replaced
select-target-size 512 --horizon 80   # design is [512(updated), 1024]
select-target-size --auto       # merges the recommendation in the same way
select-target-size --reset      # design is []
```

A size that is not yet in the design is appended. A size that is already in it has its complete entry replaced in place, keeping its position, so revising one size never disturbs another. `--reset` clears the whole design; it works only before the freeze, and it keeps your prepared generation and any diagnostic evidence you already paid for.

You can change your mind as often as you like until `cross-validate` freezes the whole design at once. After that, `select-target-size` refuses to change anything: starting a different experiment means a fresh `prepare` generation.

Everything downstream then gains a size dimension and nothing else. `cross-validate` validates the method for **every** selected size and is accepted only if all of them pass; `train-production` refuses to start **any** production run until every selected size has accepted cross-validation, and then trains and publishes one final product per size. Nothing ever picks a winner among the sizes for you.

The two training horizons

The same command steers how long downstream training runs:

```
select-target-size 512 --horizon-cv 20 --horizon 60
```

`--horizon-cv` is the cross-validation max epochs; `--horizon` is the final-production max epochs. They are independent controls, and the software deliberately does not infer either one from `N`: no proven target-size-to-horizon scaling law exists here.

Both apply to the size *this* invocation touches, so different selected sizes can carry different budgets. Omit a flag and it resolves, *on that invocation*, from your configuration - `[post_selection.cv].max_num_epochs` (default 30) and `[training].max_num_epochs` (default 30). The resolved numbers are then stored with that entry, so editing `campaign.toml` afterwards does not silently rewrite a decision you already made, and sizes you did not touch keep the numbers they were given. Reselecting a size re-resolves any flag you omit that time. An override applies to that invocation only; it never becomes a sticky default, and the CLI never rewrites `campaign.toml`.

The optional automatic diagnostic

`--auto` runs the paired optimizer-seed screen. The screen uses the configured power range, direct nested evaluation populations `M1 subset M2 subset M3`, the configured `fidelity_epochs`, and the ordered seeds from the sole enabled training method. It runs the authenticated continuation:

```
n1 / M1 -> n2 / M2 -> n3 / M3
```

The current default is `(n1, n2, n3) = (1, 3, 10)`. Boundaries are continuation points; an earlier better checkpoint cannot replace the prescribed endpoint. The reducer first narrows the qualified population, then either **recommends** one size or reports that it could not make a valid comparison. It freezes nothing either way.

Read the recommendation for what it is. The screen measures target-force RMSE after 1, 3 and 10 epochs under one configured protocol. It is not evidence that the size is asymptotically converged, that training for 60 epochs would rank the sizes the same way, or that the resulting potential is stable in MD. Treat it as one input to your decision alongside your own judgement about cost and risk.

Every completed diagnostic writes a portable Markdown report under `results/` containing the full per-boundary, per-seed evidence, the survivor progression, the normalization geometry, and the recommendation or the reason there is none. It is meant to be read, and it is safe to delete: it is a rebuildable projection, never authority.

Re-running `--auto` on an unchanged campaign is cheap. It authenticates the cached diagnostic, reruns no training or evaluation at all, and says so.

Larger candidates take more optimizer steps per epoch, so the screen normalizes learning-rate amplitude and EMA decay against a reference size. You configure the reference point, not the per-candidate values:

```
[target_data.size_convergence.optimizer_normalization]
reference_target_size = 1024
reference_learning_rate = 1.0e-4
reference_ema_decay = 0.99999
```

A candidate with twice the reference update geometry runs at half the learning rate and the square root of the EMA decay, so a size comparison is not also an optimizer-progress comparison. Epoch counts, batch size, LR schedule shape, and every other optimizer setting are identical across candidates. The reference size does not have to be one of the candidates. This applies to the screen only: `cross-validate` and `train-production` start fresh under their own method policy. Changing a reference value invalidates screen trajectories and requires a fresh screen; it does not invalidate `prepare`.

The loss the screen optimizes is the objective you configured:

```
[objective]
energy_weight = 1.0
forces_weight = 10.0
stress_weight = 1.0
```

These global coefficients are written into every generated MACE configuration - for the screen, for cross-validation, and for final production - so MACE's own `forces_weight = 100` default never applies. They are separate from `[weighting]` (the per-configuration weight) and from the per-frame property weights, which only mark whether a label is present. Editing `[objective]` changes preparation identity, so it requires a fresh `prepare`.

Replay metrics, post-selection CV, physical-observable evidence, and downstream qualification cannot rank or tie-break a size.

The configured ceiling is a **practical budget limit**, not a requirement that convergence happen below it. If the largest configured size is still materially better than every other finalist, that size is recommended and `status` reports the warning `nonconverged_at_configured_ceiling`.

Read that as: this is the best size available within your configured budget, and the screen did not show a plateau below it. No rescue size outside the configured ladder is invented. If you want to know whether a larger dataset would help, raise `target_size_power_max` and run a fresh screen.

Inside the practical-equivalence band the smaller size is still preferred, so a tiny improvement at the ceiling is treated as a plateau, not a warning.

A diagnostic that simply lacks enough comparable candidates reports **no recommendation**. That is a conclusion about the diagnostic, not about your campaign: the command succeeds, your existing provisional choice is left exactly as it was, and you can still select any qualified candidate explicitly and proceed. An incomplete but nonterminal run remains resumable, and resuming it never destroys a choice you already made.

The membership of any chosen size is not an editable field. Every read re-derives it from `pi_train` through the P2 training order; divergence fails closed.

5. Freeze the design and validate the method

```
python tools/mdstats-mlff-campaign.py --config campaign.toml cross-validate
```

`cross-validate` is the freeze point. It admits your **complete** provisional design at once - every selected size, its exact membership, and both of its horizons - and makes it immutable ancestry before any training starts. If any selected size fails to authenticate, nothing is frozen and no training begins. After this, `select-target-size` refuses to change the design; starting a different experiment means a fresh `prepare` generation.

It then runs, for every frozen size in the order you selected them, exactly the same cross-validation as before on that size's own `T_N` and its own CV horizon. It validates the training method, not the amount of data. The configured `K >= 2` folds preserve the P1 split-exclusion and correlation relations; every required fold and optimizer seed must pass the target-only acceptance predicate. Cross-validation succeeds only if **every** selected size passes; a size that fails stays visibly failed and is never dropped from your design.

Fold partitions are constructed inside the already frozen selected set. A fold may fit training-only transforms from its own training partition, freezes its representative on its authorized monitor, and evaluates the held-out partition only afterwards. Held-out fold results cannot change `N_selected`, membership, checkpoint policy, or the method definition. Replay remains a separate admissibility/retention concern and supplies no ranking credit.

A missing or failing fold is a methodological failure. It leaves selection evidence unchanged and does not authorize final production; it is not replaced by a mean, majority, best-seed, or partial-fold result.

6. Train fresh final production

```
python tools/mdstats-mlff-campaign.py --config campaign.toml train-production
```

`train-production` first checks the **whole** design: every selected size must have current accepted cross-validation evidence of its own. If any does not, it starts no production run at all - for any size - and tells you which sizes are blocking. That is deliberate: an experiment you asked for across several sizes must not quietly become the subset that happened to work.

Then, for each selected size, final production starts from the accepted foundation with fresh optimizer, RNG, and run state. It trains that size's complete exact `T_N` using the method accepted by cross-validation for that size and that size's own frozen production horizon. Each size publishes its own final product; no size can consume another size's data, evidence, or publication. Screening and CV checkpoints are not production parents, even when their numeric seed or size matches.

The production horizon is independent of the screen's `n3`. A production-only configuration change invalidates production descendants while leaving the selected binding and accepted CV evidence current. Both post-selection owners re-authenticate currentness before work and publish only under a commit-time currentness fence.

`train-production` finishes by publishing the **final-production publication decision**: which of the completed seeds constitute the released product, under the configured `[post_selection.production].committee_policy`. Both policies are supported. `all_qualified_final_seeds` publishes every required seed whose already-frozen representative checkpoint is admissible; `single_best_final_seed` ranks those same already-frozen representa-

tives with the accepted target-only EVAL2 ordering over the frozen M3 development evidence and publishes one. The decision is taken here, before any qualification evidence exists, and nothing downstream can change it.

The training lifecycle ends at that publication. Everything after it validates the finished product without being able to change it.

If you selected more than one size

When your design has several sizes, `train-production` finishes with several final products, and the training experiment is then **complete but not release qualified**. `advance` stops there and offers no further command, and `qualification status` explains why: nothing in this revision is authorized to decide which of your products is *the* release, and picking the first, the last, the diagnostic's recommendation, or the best-scoring one would be that decision made silently. `qualification run` and `qualification activate-locked` therefore fail closed before they create an attempt or open any locked evidence - in particular, one-shot locked data is never spent comparing sizes.

Compare the per-size results reported by `status`, decide for yourself, and then qualify that product in its own campaign: a fresh `prepare` generation with exactly one selected size. Everything in the next section applies to that single-size case.

7. Qualify the frozen product

Qualification consumes the final-production publication that `train-production` already froze. It never creates, reorders, or shrinks that publication, and it owns no target-size, cross-validation, production, checkpoint, seed, or member decision. Every threshold under `[qualification]` is fixed in the configuration before any product outcome is observed.

```
python tools/mdstats-mlff-campaign.py --config campaign.toml qualification status
python tools/mdstats-mlff-campaign.py --config campaign.toml qualification run
```

`qualification run` executes or resumes the nonlocked components for the exact frozen publication:

- **deployment parity** - the published model is exported at the canonical target head, converted to the deployed ML-IAP artifact at that same head, executed through the real supported LAMMPS runtime, and compared against the authenticated in-framework model under dtype-justified tolerances, including stress whenever the product actually has a stress channel. Stress applicability is decided from the accepted training objective, the reference labels, the model, periodicity, and the runtime - not from a configuration switch, so `stress_required` can insist on qualifying an available channel but nothing can quietly suppress one;
- **local PES** - deterministic symmetric displacement modes on a candidate-independent `OUTER_MONITOR` base cohort, checked for pointwise force agreement, restoring sign, and stiffness/curvature against matched external references;
- **relaxation** - fixed-cell minimization compared against matched reference relaxations, with protected-topology safety judged separately from geometric fidelity;
- **dynamics** - bounded NVT warm-up and NVE propagation through the deployed artifact, started from the authenticated reference-relaxed geometry of each physical base, and checked for NVT and NVE temperature behaviour, energy drift, minimum pair distance, force bounds, and protected topology, displacement, bond, and angle degradation. Topology damage must persist for a configured number of consecutive samples before it rejects, so one noisy sample is not mistaken for a broken framework;
- **calibration** - uncertainty calibration of the exact frozen committee on the reserved `UNCERTAINTY_CALIBRATION` role, or an explicit `not_applicable` for a single-model product with no accepted uncertainty estimator.

Three outcomes are not failures and must not be read as one:

- `waiting_for_reference` means the external first-principles evidence the frozen physical plan asked for has not been supplied. The exact request is written to the reference root as `reference-request.json`; supply a matching `reference-bundle.json` and rerun. Nothing is ever passed on absent evidence.
- `not_applicable` means the frozen policy declares a component inapplicable to this product.
- an *unavailable* supported deployment runtime blocks the deployment claim rather than passing or rejecting it. Executing *an* ML-IAP model and executing *this MACE product* are separate capabilities, and only the second one proves the product path; `qualification status` reports both.

`[qualification.reference].protocol` must name a real reference method before any reference-dependent component runs. The generated placeholder fails closed rather than letting an unlabelled bundle become a release claim.

A component rejection rejects that exact published product. It never selects a different seed, checkpoint, or committee member, never shrinks a committee, and never reaches back into target-size selection, cross-validation acceptance, or production training.

The one-shot locked test

```
python tools/mdstats-mlff-campaign.py --config campaign.toml qualification activate-locked --
confirm
```

This is the only path that opens the reserved `LOCKED_INTERPOLATION_TEST` cohort. It requires `--confirm`, requires every mandatory nonlocked component to have already passed, and is refused a second time for the same publication and cohort. Activation is irreversible: once the cohort is revealed it is never a fresh locked test again, whatever the policy is later changed to, and a retrained product needs genuinely new independent evidence.

A locked pass produces the terminal `release_qualified` verdict; a locked failure rejects the exact published product.

Activation is an irreversible *open* event, not a claim that the evaluation finished. If the process dies between opening the cohort and publishing the result, rerunning `activate-locked --confirm` resumes the same activation and finishes the same test; it never opens a second one. Only a genuinely completed terminal result makes a further activation a rejected duplicate.

Supplying a new reference bundle for the same frozen request re-runs local PES, relaxation, and dynamics - the components that consume it - and reuses the deployment and calibration evidence, which do not.

Where the evidence lives

```
campaign/.mdstats/qualification/g<N>/objects/    immutable release evidence
campaign/.mdstats/qualification/g<N>/attempts/   attempt state and scratch
campaign/qualification-references/<plan>/        reference request and bundle
```

Durable qualification evidence is release evidence, not reconstructible scratch: storage cleanup never reclaims it, and it also cannot reclaim an artifact an in-flight qualification attempt still references.

Each attempt also records what it actually cost - elapsed time overall and per component, workspace free space and the attempt's own footprint at start and end, peak memory, and accelerator telemetry where available - and the terminal record points at it. Before each component materializes artifacts or scratch, the campaign's existing `[execution].minimum_free_disk_gib` reserve is checked; an attempt that cannot proceed safely aborts rather than changing anything scientific. Mixed periodic boundaries are executed exactly as configured, so a `[True, True, False]` system runs as itself rather than being silently coerced.

Inspect and resume

```
python tools/mdstats-mlff-campaign.py --config campaign.toml status
python tools/mdstats-mlff-campaign.py --config campaign.toml advance
```

The workspace stores the durable campaign state and content-addressed payloads:

```
campaign/
|-- campaign-manifest.json
|-- .mdstats/campaign.sqlite3
|-- .mdstats/                    # current-generation records and caches
|-- runs/                         # authorized checkpoints and logs
|-- models/                       # current production model evidence
`-- results/                      # bounded summaries and cleanup reports
```

Rerunning a current owner is safe: complete authenticated cells are reused, stale or corrupt derived caches are rebuilt, and a superseded owner cannot publish a current descendant. Close stores/processes cleanly before inspecting or rerunning another owner.

Storage management

Storage is orthogonal to the scientific lifecycle. What may be touched is decided by the real P1-P7 owners - never by a pathname, a report label, a stage name, a file age, or a process id. Start with a read-only report:

```
python tools/mdstats-mlff-campaign.py --config campaign.toml storage report
python tools/mdstats-mlff-campaign.py --config campaign.toml storage report --deep
```

`storage report` reads the owner views and bounded physical metadata; it tells you which owner holds each family, what is current, and how much each action could reclaim. It is bounded - one `lstat` per owner artifact, no subtree walk - so a directory's total size is reported as unknown rather than guessed, and `--deep` is the explicit opt-in to an exact recursive physical audit.

Both are read-only, and read-only here means the command changes nothing at all: it does not create the workspace or the campaign database, it writes no report file into `results/`, and it does not even warm the SHA-256 receipt cache. The campaign database is opened read-only at the SQLite level for the whole invocation - including from any worker thread it uses - so this is enforced, not just intended. The report is printed, not deposited. Running it against a directory that was never prepared tells you the campaign is uninitialized instead of initializing it. A read-only command running at the same time as real campaign work does not disturb it.

Authority to change anything comes only from the invocation you are running. `--apply` on this command line authorizes the mutation and the subcommand you typed selects the action; a persisted `apply` or `action` key under `[storage]` in your TOML is rejected outright rather than obeyed. Every consequential action plans first and mutates only when you authorize it:

```
python tools/mdstats-mlff-campaign.py --config campaign.toml storage cleanup --tier safe --dry-
run
python tools/mdstats-mlff-campaign.py --config campaign.toml storage cleanup --tier safe --
apply
python tools/mdstats-mlff-campaign.py --config campaign.toml storage cleanup --tier cache --
dry-run
python tools/mdstats-mlff-campaign.py --config campaign.toml storage cleanup --tier cache --
apply
```

`--tier safe` loses no scientific, restart, qualification, locked, or acceleration-cache capability: it removes only artifacts an owner has positively released, such as external record payloads no campaign state row references. `--tier cache` adds eviction that an owner can certify as exactly reconstructible right now *and* can certify nothing is currently depending on. Reconstructibility alone is not enough. The normalized frame cache is reported as exactly reconstructible - rebuilding costs one source read per DATA2 run and reproduces the identical authenticated cache - but P1 exposes no liveness seam that could prove no reader is using it at this instant, so it is retained by both tiers and reported as `cache_reconstructible = true, cache_evictable = false`. Its members are also

content-addressed and shared: two prepared generations that differ in one run reference the same bytes for every run the change did not touch, so a member is protected while *any* prepared generation still needs it, never merely because of the directory it sits in. The immutable prepared substrate under `.mdstats/prepared` is reported the same way, as restart state the current generation requires. The SHA-256 receipt store is likewise accounted as a cache and retained. Anything no owner can certify is retained, so a `cache` run today is legitimately a no-op on most campaigns.

Cleanup also plans campaign-state maintenance as its own actions rather than as a side effect, and pruning and rewriting are two of them. Too many diagnostic events authorize *pruning*, down to exactly the number you configured; they never authorize a full database rewrite. The rewrite is planned only when SQLite's own free-space accounting already says it is worth its cost and there is room for the temporary copy it makes - so if pruning is what frees the space, the rewrite waits for your next `storage cleanup`. Before rewriting, it excludes every other campaign writer, including one in another process, and rechecks; if a concurrent run consumed the free space while it waited, it skips the rewrite rather than doing it anyway. The result tells you which of the two actually happened.

Cold archive turns owner-declared *historical* bulk into a reversible, authenticated, identity-keyed cold representation:

```
python tools/mdstats-mlff-campaign.py --config campaign.toml storage archive create --dry-run
python tools/mdstats-mlff-campaign.py --config campaign.toml storage archive create --apply
python tools/mdstats-mlff-campaign.py --config campaign.toml storage archive list
python tools/mdstats-mlff-campaign.py --config campaign.toml storage archive verify <identity>
python tools/mdstats-mlff-campaign.py --config campaign.toml storage archive restore <identity>
--apply
```

Hot bytes are removed only after the archive is authenticated and cataloged, and only for artifacts no current or restartable owner needs hot - a checkpoint the current qualification publication still authenticates is never archived out from under it. Only descendants the owning component certifies as its own are collected, and the check is by node *kind* as well as by name: a file something else dropped inside an otherwise eligible run tree, an empty directory nobody recorded, a symlink, or a file swapped for a directory at the same path all withhold authority over that whole tree until you clear them. The same applies to a released qualification attempt's leftover scratch.

If a qualification attempt's own state cannot be authenticated, `storage report` says so and every consequential command refuses until you repair it - not just commands touching qualification. That attempt may have been pinning exact checkpoints anywhere in the campaign, and the product will not guess which, so nothing campaign-managed is authorized for deletion while the ambiguity lasts. Reporting deliberately keeps working: that is when you need it. Repairing the exact state restores normal behavior with no migration and no guessing.

`--root` may narrow the selection to one eligible artifact. It may not widen it: naming a parent directory is rejected rather than reinterpreted, because a parent sweeps in siblings no owner released.

Restoring an archive brings the bytes back as *historical* evidence; it never promotes anything to current, and it never changes the permissions or ownership of a directory that already existed.

Both `restore` and `reclaim` re-authenticate the exact archive they planned against at the last possible moment, while holding the storage lock. If the blob, manifest, or catalog entry changed or vanished after you planned - a bad disk, an interrupted copy - nothing is deleted and nothing is installed; you re-plan instead. If a reclamation was interrupted, `storage archive reclaim <identity> --apply` resumes it against a freshly authenticated archive, including when the interruption already removed the run's own evidence file.

Deduplication is a representation change only:

```
python tools/mdstats-mlff-campaign.py --config campaign.toml storage deduplicate --dry-run
python tools/mdstats-mlff-campaign.py --config campaign.toml storage deduplicate --apply
```

It shares one inode directly between byte-identical files whose owner certifies both immutability and matching filesystem metadata. There is no separate content store, so deleting the last remaining name releases the bytes with nothing left to collect. Equal bytes alone are never enough; a file already hardlinked to something outside the campaign is never chosen as the shared inode, and a cross-device or unsupported filesystem simply keeps the duplicates.

Every applied action normally leaves one durable record in the storage audit, and bounded retention of that log happens in the same serialized step, so a concurrent operation can never trim away a record that was just published. If the audit itself cannot be written - a full disk, an I/O error - the command does not pretend otherwise and does not undo work that already succeeded: it reports the outcome as `..._unaudited` and prints the publication failure, so you know the change happened but the durable record of it did not. If only the *trimming* fails, the record still stands and the command says so; the next operation retries it.

Every action protects external inputs, current scientific records, selected checkpoints, restart evidence, and diagnostics. The retired `recompute` and `compact` loss tiers are not current product authority and are rejected by name.

Interpreting outcomes and limitations

The durable result is the authenticated chain of source identity, neutral substrate, target-size experiment, selected binding, CV acceptance, and final production identity. An automatic-diagnostic no-recommendation outcome is diagnostic evidence: it does not block an explicit target-size choice, and it does not by itself expose a production next action. A missing accelerator, unavailable target-machine run, or absent downstream qualification is reported as deferred or unavailable rather than silently passed.

The P6 implementation provides current functional, restart, and public-surface closure. It does not establish GPU, long real-data, M-ladder decision- preservation, or downstream release qualification.